DATA 412/612 · Statistical Programming in R · Week 9

9.5 Regex: Groups and Lookaheads

Parentheses group a pattern. A capturing group also remembers what it matched. A lookahead only checks what comes next.

By the end of this note you should be able to:

1. Explain what parentheses do in a regular expression.
2. Distinguish a capturing group `(...)` from a non-capturing group `(?:...)`.
3. Read the matrix that `str_match()` returns: the whole match, then each captured group.
4. Number capturing groups from their opening parentheses.
5. Use alternation (`|`) inside a group.
6. Use a positive lookahead `(?=...)` and a negative lookahead `(?!...)`.
7. Tell a lookahead, which only checks, from a pattern that consumes characters.
8. Apply these ideas to realistic text.

1. Pattern matching in R

`stringr` has several functions for text patterns.

```
library(stringr)
```

Function	Purpose
`str_detect()`	Does the pattern occur?
`str_extract()`	First matching substring
`str_extract_all()`	Every matching substring
`str_match()`	The whole match and its capturing groups
`str_match_all()`	Every match and its capturing groups
`str_replace()`	Replace the first match
`str_replace_all()`	Replace every match
`str_count()`	Count non-overlapping matches

```
x <- "John123"

str_detect(x, "[A-Za-z]+")
str_extract(x, "[A-Za-z]+")
```

```
[1] TRUE
[1] "John"
```

`str_detect()` says whether a match exists. `str_extract()` returns the matched text.

Sometimes you want the pieces of one match separately. That is what a capturing group is for.

2. A group is parentheses

Parentheses group part of a pattern.

```
"(ab|bcs)"
```

This matches `"ab"` or `"bcs"`. The symbol `|` means OR.

```
str_detect(c("ab", "bcs", "abc", "dog"), "(ab|bcs)")
```

```
[1] TRUE TRUE TRUE FALSE
```

`"abc"` matches because it contains `"ab"`.

To match only the complete strings `"ab"` or `"bcs"`:

```
str_detect(c("ab", "bcs", "abc", "dog"), "^(ab|bcs)$")
```

```
[1] TRUE TRUE FALSE FALSE
```

`^` is the start of the string. `$` is the end.

Ordinary parentheses `(...)` do two things: they group, and they capture. Capturing is next.

3. Capturing groups `(...)`

A capturing group matches a piece and keeps that piece.

```
x <- "John123"

str_match(x, "([A-Za-z]+)([0-9]+)")
```

```
     [,1]      [,2]   [,3]
[1,] "John123" "John" "123"
```

`([A-Za-z]+)` is one or more English letters. The parentheses capture them. This group matches `"John"`.

`([0-9]+)` is one or more digits, captured. This group matches `"123"`.

Together, `"([A-Za-z]+)([0-9]+)"` requires letters followed immediately by digits, so the whole match is `"John123"`.

Column	Output	Meaning
`[,1]`	`"John123"`	Entire match
`[,2]`	`"John"`	Capturing group 1
`[,3]`	`"123"`	Capturing group 2

The first column is always the entire match. Each later column is one capturing group.

`str_match()` returns one more column than the number of capturing groups. R does not search three times. The engine matches the pattern once and records the groups as it goes.

4. Why capture

Student identifiers:

```
students <- c("John123", "Alice45", "Bob789")

str_match(students, "([A-Za-z]+)([0-9]+)")
```

```
     [,1]      [,2]    [,3]
[1,] "John123" "John"  "123"
[2,] "Alice45" "Alice" "45"
[3,] "Bob789"  "Bob"   "789"
```

```
matches <- str_match(students, "([A-Za-z]+)([0-9]+)")
names <- matches[, 2]
numbers <- matches[, 3]
```

```
names
```

```
[1] "John" "Alice" "Bob"
```

```
numbers
```

```
[1] "123" "45" "789"
```

Those numbers are still character strings. `as.numeric()` turns them into numbers when you need that.

5. Non-capturing groups `(?:...)`

A non-capturing group groups without keeping a separate column:

```
(?:...)
```

The `?:` sits immediately after the opening parenthesis. It is not applied to a group that already exists.

Both groups captured:

```
str_match("John123", "([A-Za-z]+)([0-9]+)")
```

```
     [,1]      [,2]   [,3]
[1,] "John123" "John" "123"
```

First group not captured:

```
str_match("John123", "(?:[A-Za-z]+)([0-9]+)")
```

```
     [,1]      [,2]
[1,] "John123" "123"
```

`(?:[A-Za-z]+)` still matches `"John"`. It does not get a column. `([0-9]+)` matches `"123"` and is capturing group 1, even though it is the second pair of parentheses in the pattern. Non-capturing groups are not numbered.

Column	Output	Meaning
`[,1]`	`"John123"`	Entire match
`[,2]`	`"123"`	First capturing group

Second group not captured:

```
str_match("John123", "([A-Za-z]+)(?:[0-9]+)")
```

```
     [,1]      [,2]
[1,] "John123" "John"
```

The digits are matched and not stored.

Neither group captured:

```
str_match("John123", "(?:[A-Za-z]+)(?:[0-9]+)")
```

```
     [,1]
[1,] "John123"
```

No capturing groups, so `str_match()` returns only the whole match.

Pattern	Capturing groups	Columns
`([A-Za-z]+)([0-9]+)`	2	3
`(?:[A-Za-z]+)([0-9]+)`	1	2
`([A-Za-z]+)(?:[0-9]+)`	1	2
`(?:[A-Za-z]+)(?:[0-9]+)`	0	1

6. How capturing groups are numbered

Capturing groups are numbered by their opening parentheses, left to right. A non-capturing group gets no number.

```
x <- "John-123-USA"

str_match(x, "([A-Za-z]+)-(?:[0-9]+)-([A-Za-z]+)")
```

```
     [,1]          [,2]   [,3]
[1,] "John-123-USA" "John" "USA"
```

Group in the pattern	Capturing?	Number
`([A-Za-z]+)`	Yes	1
`(?:[0-9]+)`	No	none
`([A-Za-z]+)`	Yes	2

Groups can be nested. The outer pair counts too.

```
str_match("abc123", "((abc)(123))")
```

```
     [,1]     [,2]     [,3]  [,4]
[1,] "abc123" "abc123" "abc" "123"
```

Column 1 is the whole match. Group 1 captures `"abc123"`. Group 2 captures `"abc"`. Group 3 captures `"123"`.

7. Capturing and non-capturing side by side

```
x <- "red-blue"

str_match(x, "(red)-(blue)")
```

```
     [,1]       [,2]  [,3]
[1,] "red-blue" "red" "blue"
```

```
str_match(x, "(?:red)-(blue)")
```

```
     [,1]       [,2]
[1,] "red-blue" "blue"
```

The text that matches does not change. Only the captured columns change.

Use `(...)` when you want that piece back. Use `(?:...)` when you need a group for `|` or for a quantifier, and you do not need the piece as its own column.

```
str_detect(c("cat", "dog", "bird"), "^(?:cat|dog)$")
```

The group lets `^` and `$` apply to both alternatives. `str_detect()` only returns `TRUE` or `FALSE`, so there is nothing to capture.

8. A lookahead checks and does not consume

A capturing or non-capturing group matches text. A lookahead only checks the text at the current position. It does not include that text in the match, and it does not move forward.

Syntax	Name	Meaning
`(?=...)`	Positive lookahead	This pattern must appear here
`(?!...)`	Negative lookahead	This pattern must not appear here

9. Positive lookahead `(?=...)`

```
x <- c("cat123", "dog456", "catABC")

str_extract(x, "cat(?=[0-9])")
```

```
[1] "cat" NA NA
```

`cat` matches the letters. `(?=[0-9])` checks that the next character is a digit and does not take the digit.

```
str_extract("cat123", "cat(?=[0-9])")
```

```
[1] "cat"
```

That is `"cat"`, not `"cat1"`. Compare a pattern that does consume the digit:

```
str_extract("cat123", "cat[0-9]")
```

```
[1] "cat1"
```

10. Negative lookahead `(?!...)`

```
x <- c("cat", "cats", "dog", "catalog", "bird")

str_detect(x, "^(?!cat).*$")
```

```
[1] FALSE FALSE TRUE FALSE TRUE
```

The pattern `"^(?!cat).*$"` has four pieces.

1. `^` is the start of the string.
2. `(?!cat)` checks that the next characters are not `"cat"`.
3. `.*` matches the rest of the characters.
4. `$` is the end of the string.

On `"cat"`, the next three characters are `"cat"`, so the lookahead fails and the match fails.

On `"dog"`, the next characters are not `"cat"`. The lookahead succeeds, and `.*` takes the rest.

On `"catalog"`, the string starts with `"cat"`. The extra letters do not save it. The lookahead fails.

11. Excluding a whole string or a substring

```
x <- c("ab", "bcs", "abc", "hello", "xxabyy", "dog")
```

Reject only the complete strings `"ab"` and `"bcs"`:

```
str_detect(x, "^(?!(?:ab|bcs)$).+$")
```

```
[1] FALSE FALSE TRUE TRUE TRUE TRUE
```

`^` is the start. `(?!(?:ab|bcs)$)` checks that what remains is not exactly `"ab"` or `"bcs"`. The `$` inside the lookahead means the alternative has to reach the end. `(?:ab|bcs)` groups the two alternatives and does not capture them. Then `.+$` matches one or more characters through the end, so the empty string is rejected too.

Reject any string that contains `"ab"` or `"bcs"` anywhere:

```
str_detect(x, "^(?!.*(?:ab|bcs)).*$")
```

```
[1] FALSE FALSE FALSE TRUE FALSE TRUE
```

`.*` inside the lookahead allows any characters before the forbidden piece, so the check looks through the whole string.

Input	Exclude the exact string	Exclude if it contains the piece
`"ab"`	FALSE	FALSE
`"bcs"`	FALSE	FALSE
`"abc"`	TRUE	FALSE
`"hello"`	TRUE	TRUE
`"xxabyy"`	TRUE	FALSE
`"dog"`	TRUE	TRUE

12. Often `!str_detect()` is enough

For many problems you do not need a negative lookahead. `!` reverses `str_detect()`.

```
x <- c("ab", "bcs", "abc", "hello", "xxabyy", "dog")

!str_detect(x, "^(?:ab|bcs)$")
!str_detect(x, "ab|bcs")
```

The first line drops the complete strings `"ab"` and `"bcs"`. The second drops anything that contains either piece. This is usually easier to read.

One difference: `!str_detect(x, "^(?:ab|bcs)$")` accepts an empty string. The lookahead pattern in section 11 uses `.+`, so it rejects the empty string.

13. `(...)`, `(?:...)`, `(?=...)`, and `(?!...)`

Syntax	Name	Matches text?	Captures?
`(...)`	Capturing group	Yes	Yes
`(?:...)`	Non-capturing group	Yes	No
`(?=...)`	Positive lookahead	Checks only	No
`(?!...)`	Negative lookahead	Checks only	No

```
x <- "cat123"

str_match(x, "(cat)")
str_match(x, "(?:cat)")
str_extract(x, "cat(?=123)")
str_extract(x, "cat(?!ABC)")
```

`(cat)` returns the whole match and `"cat"` as a captured group. `(?:cat)` returns only the whole match, which is `"cat"` (not the rest of `"cat123"`). `cat(?=123)` returns `"cat"` after checking that `"123"` follows. `cat(?!ABC)` returns `"cat"` because `"cat"` is not followed by `"ABC"`.

14. A group that does not participate

```
str_match("John", "([A-Za-z]+)([0-9]+)")
```

```
     [,1] [,2] [,3]
[1,] NA   NA   NA
```

The whole pattern fails: `"John"` has no digits after the letters.

A group can be optional.

```
str_match("John", "([A-Za-z]+)([0-9]+)?")
```

```
     [,1]   [,2]   [,3]
[1,] "John" "John" NA
```

The whole match succeeds. Group 1 captures `"John"`. Group 2 does not take part, so that column is `NA`. That is different from a group that captures an empty string. The number of columns still follows the number of capturing groups, even when one of them does not participate.

15. Backreferences

A backreference reuses text that an earlier group captured.

```
str_detect("hello hello", "^(\\w+) \\1$")
```

```
[1] TRUE
```

`(\\w+)` captures the first word. The space matches a space. `\\1` means “the text group 1 captured.” `^` and `$` require the whole string to match, so the second word has to be the same as the first.

```
str_detect("hello world", "^(\\w+) \\1$")
```

```
[1] FALSE
```

R reads the string before the regex engine does, so a backslash in the pattern is written twice: `"\\w"` and `"\\1"`.

Practice

Predict the result before you run the command.

Part A. Capturing and non-capturing groups

1. What is the output?

```
str_match("Alice456", "([A-Za-z]+)([0-9]+)")
```

2. What is the output?

```
str_match("Alice456", "(?:[A-Za-z]+)([0-9]+)")
```

3. How many columns are returned?

```
str_match("abc123", "([a-z]+)(?:[0-9]+)")
```

4. How many columns are returned?

```
str_match("abc123", "((?:abc)(123))")
```

5. Why do `"(cat|dog)"` and `"(?:cat|dog)"` agree under `str_detect()` and disagree under `str_match()`?

Part B. Lookaheads

6. What does this return?

```
str_extract("cat123", "cat(?=[0-9])")
```

7. What does this return?

```
str_extract("catABC", "cat(?=[0-9])")
```

8. What is the result?

```
str_detect("catalog", "^(?!cat).*$")
```

9. Which strings return `TRUE`?

```
x <- c("ab", "abc", "dog", "xxabyy")

str_detect(x, "^(?!.*ab).*$")
```

10. What is the difference between `"cat[0-9]"` and `"cat(?=[0-9])"`?

Part C. Longer questions

11. Predict the whole output.

```
str_match(
  "John-123-USA",
  "([A-Za-z]+)-(?:[0-9]+)-([A-Za-z]+)"
)
```

12. How many capturing groups are in `"((?:ab)(cd))(ef)"`? Applied to `"abcdef"`, what does each capturing group return?
13. Write a pattern that matches `"cat"` or `"dog"`, and captures only when the match is `"cat"`. Use `str_match()`. The input is either `"cat"` or `"dog"`.
14. Write a pattern that matches a word only when a digit comes next, and leave the digit out of the match. Use `str_extract()`.
15. Write a pattern that finds strings that do not contain `"error"` anywhere. Use `str_detect()` and a negative lookahead.
16. How many columns does this return, and what changes if the input is `"John"`?

```
str_match("John123", "([A-Za-z]+)([0-9]+)?")
```

17. Explain the difference between `"(ab|cd)"`, `"(?:ab|cd)"`, and `"(?!ab|cd)"`.
18. What is the output?

```
str_detect(
  c("hello hello", "hello world", "abc abc"),
  "^(\\w+) \\1$"
)
```

19. Match an identifier of letters followed by digits, and capture only the digits. For `"Alice123"`, the whole match is `"Alice123"` and capturing group 1 is `"123"`.
20. Suppose

```
x <- c("user123", "admin456", "guest789", "adminABC", "hello")
```

Write one pattern for `str_match()` that:

1. Matches strings of letters followed by digits.
2. Rejects strings that start with `"admin"`.
3. Captures only the digits.
4. Requires the whole string to follow that form.

Say what each group and lookahead is doing.

Key points

- `(...)` groups and captures.
- `(?:...)` groups and does not capture. The `?:` comes right after `(`.
- Capturing groups are numbered by the order of their opening parentheses. Non-capturing groups are skipped.
- `str_match()` puts the whole match in column 1 and the captured groups in the columns after that.
- `(?=...)` checks that a pattern is here and does not consume it.
- `(?!...)` checks that a pattern is not here and does not consume it.
- A lookahead does not create a capturing group.
- A non-capturing group still has to match. A lookahead only tests a condition.
- `!str_detect()` is often the simpler way to drop matches.